

Экспериментальное исследование алгоритмов для регулярных запросов

Котельникова Ксения Андреевна, 23.Б15-мм

Постановка задачи

- Исследование будет проводиться в отношении следующих задач достижимости, решаемых в предыдущих работах.
 - Достижимость между всеми парами вершин
 - Достижимость для каждой из заданного множества стартовых вершин.
- Вопросы, на которые необходимо ответить в ходе исследования.
 - Какое представление разреженных матриц и векторов лучше подходит для каждой из решаемых задач?
 - Начиная с какого размера стартового множества выгоднее решать задачу для всех пар и выбирать нужные?

Описание исследуемых решений

- В рамках работы будут исследоваться следующие алгоритмы:
 - Алгоритм на основе тензорного произведения и транзитивного замыкания
 - Алгоритм на основе multiple-source BFS

Описание набора данных для экспериментов

Запросы

- Генерировались по следующим шаблонам с использованием самых часто встречающихся меток графа

Для графов с >2 часто встречающимися метками

1. $(l1 \mid l2) * l3$
2. $(l3 \mid l2) + l1$
3. $(l1 \mid l2) (l1 \mid l3) *$
4. $l1 *$

Для графов с 2 часто встречающимися метками

1. $(l1 \mid l2) * l1$
2. $(l1 \mid l2) + l2 *$
3. $l1 \ l2 (l1 \mid l2) *$
4. $l1 *$

Описание набора данных для экспериментов

Графы

- При выборе использовались следующие критерии: соотношение вершин и ребер, количество различающихся меток, наличие нескольких различных часто встречающихся меток
 - «wc», 332 вершины, 269 ребер, 2 метки (156 и 113 ребер)
 - «atom», 291 вершина, 425 ребер, 3 часто встречающиеся метки (138, 129, 122 ребра)
 - «foaf», 256 вершин, 631 ребро, 5 часто встречающихся меток (174, 78, 75, 75, 72 ребра)

Оборудование

- Процессор — Apple M1
- ОЗУ — 8Гб
- Версия Python — 3.12.5
- ОС — macOS Sonoma 14.5

Используемые библиотеки

- Разреженные матрицы и операции над ними — `scipy.sparse`
- Обработка данных экспериментов — `pandas`
- Отрисовка графиков результатов — `matplotlib.pyplot`

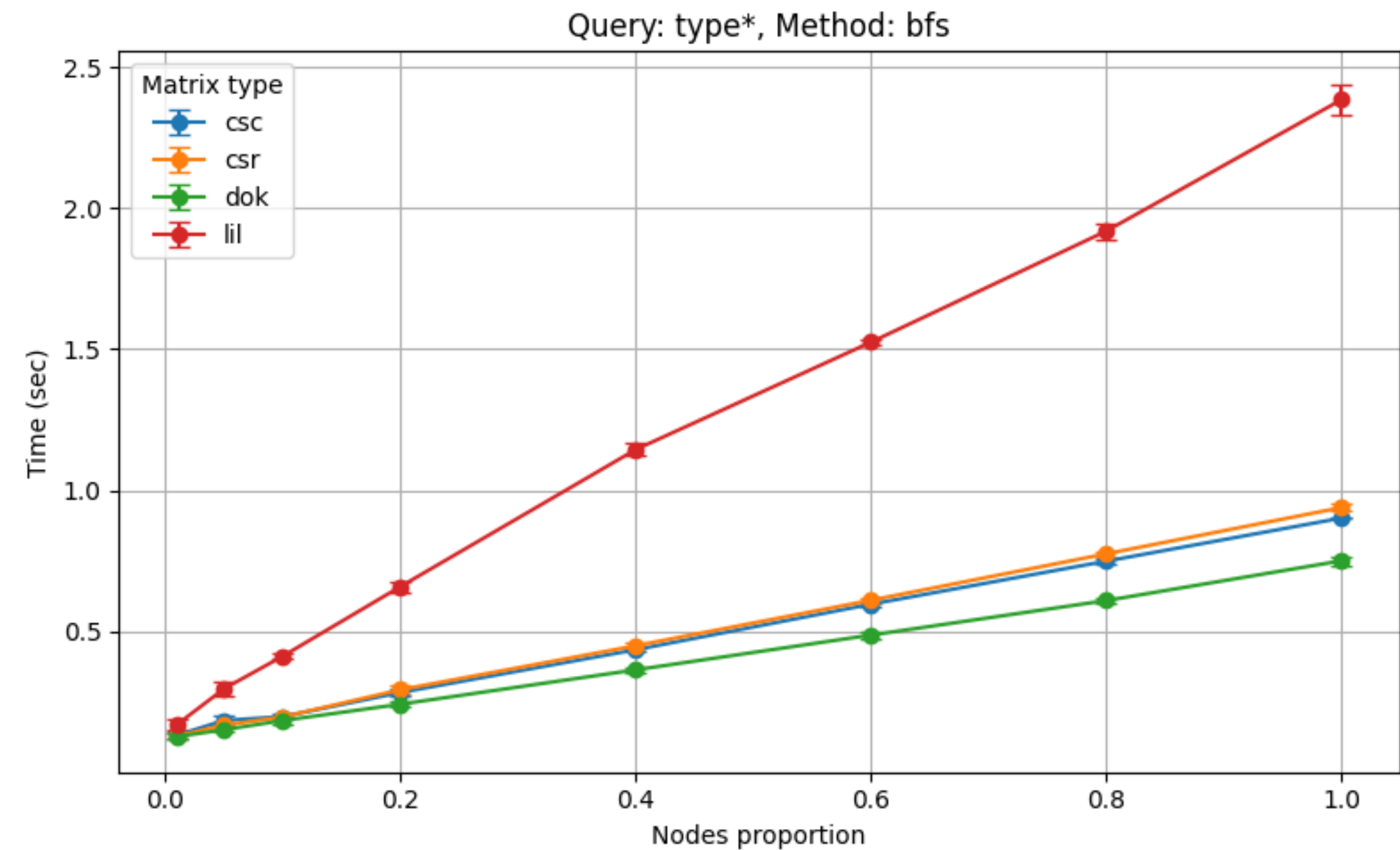
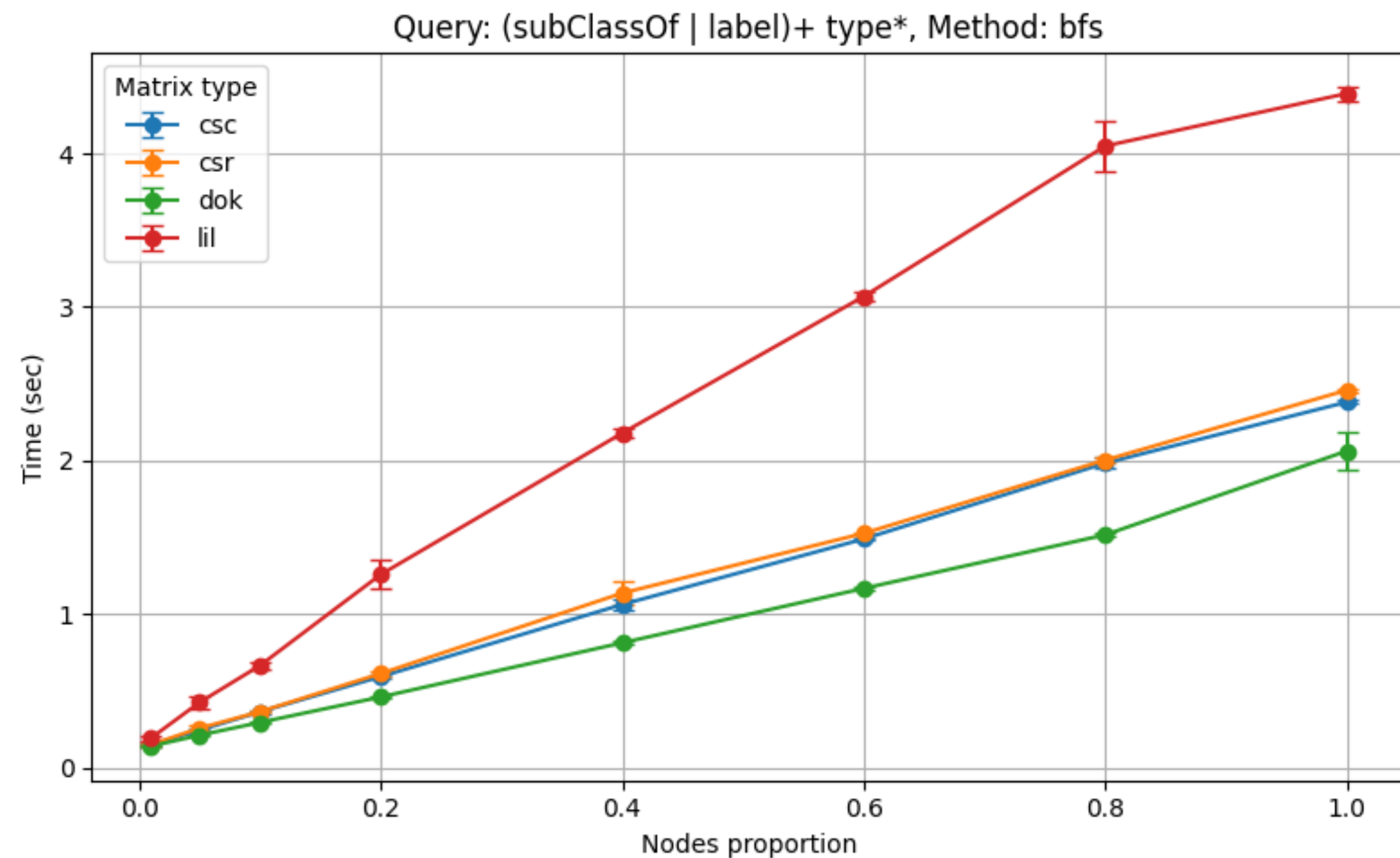
Описание проведения эксперимента

Выбор вершин

- В качестве стартовых и финальных использовались следующие доли вершин графа: 0.01, 0.05, 0.1, 0.2, 0.4, 0.6, 0.8, 1.0.
- Для неполных множеств выбирались вершины, между которыми есть пути. Если фактически вершин оказывалось существенно (на половину) меньше, чем требовалось, эксперимент на этом запросе пропускался

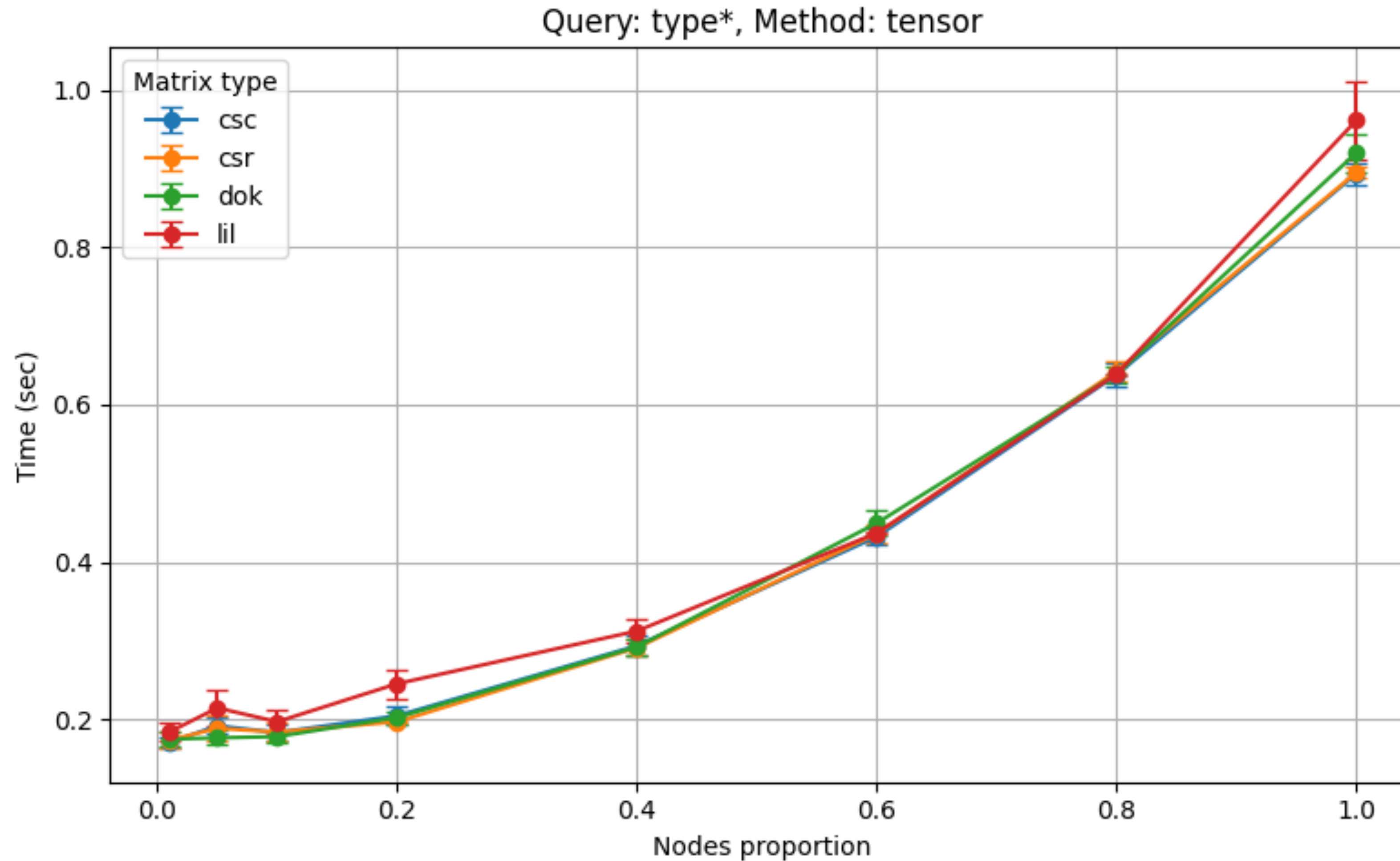
Результаты

«atom», сравнение форматов матриц



Результаты

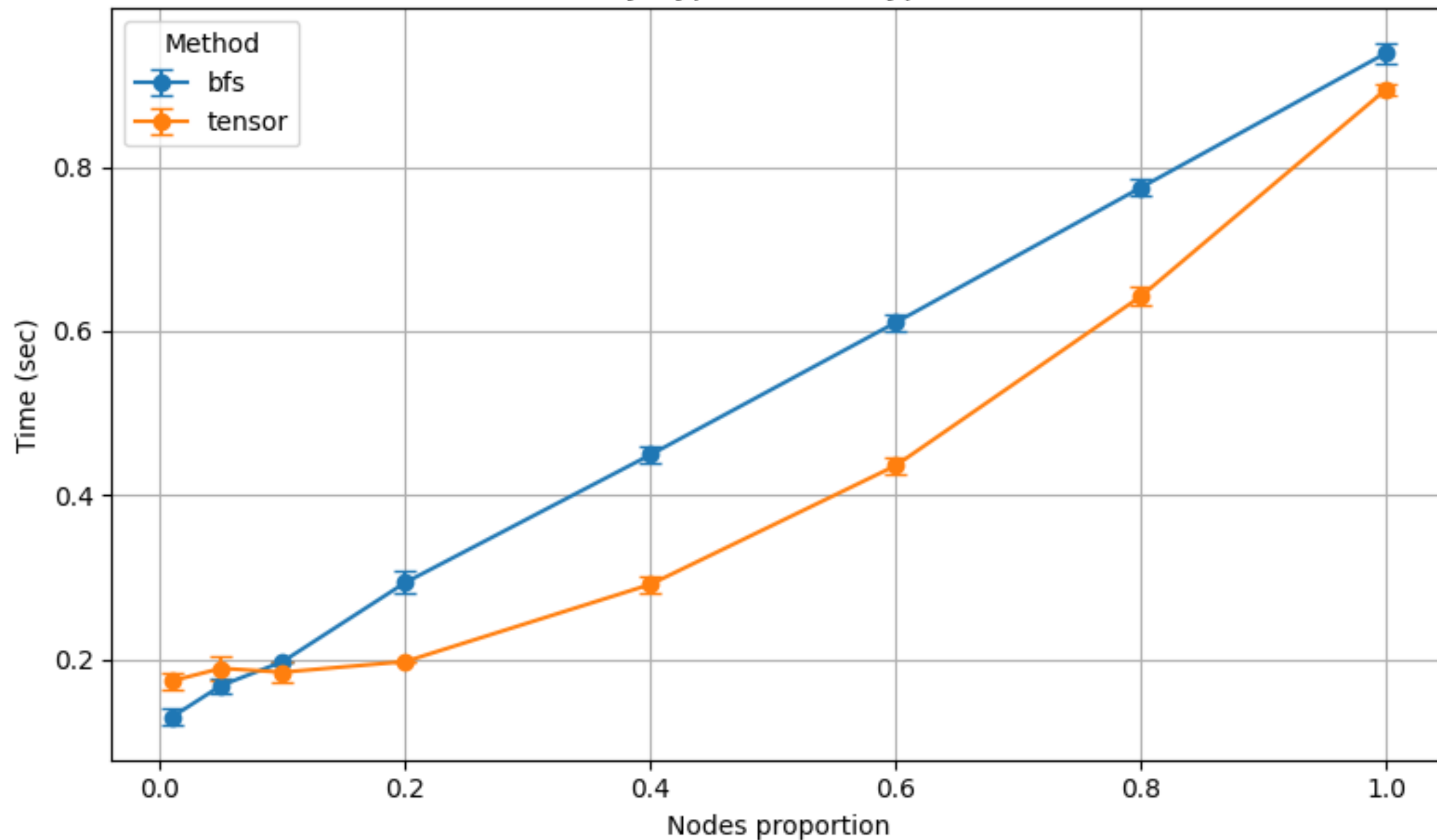
«atom», сравнение форматов матриц



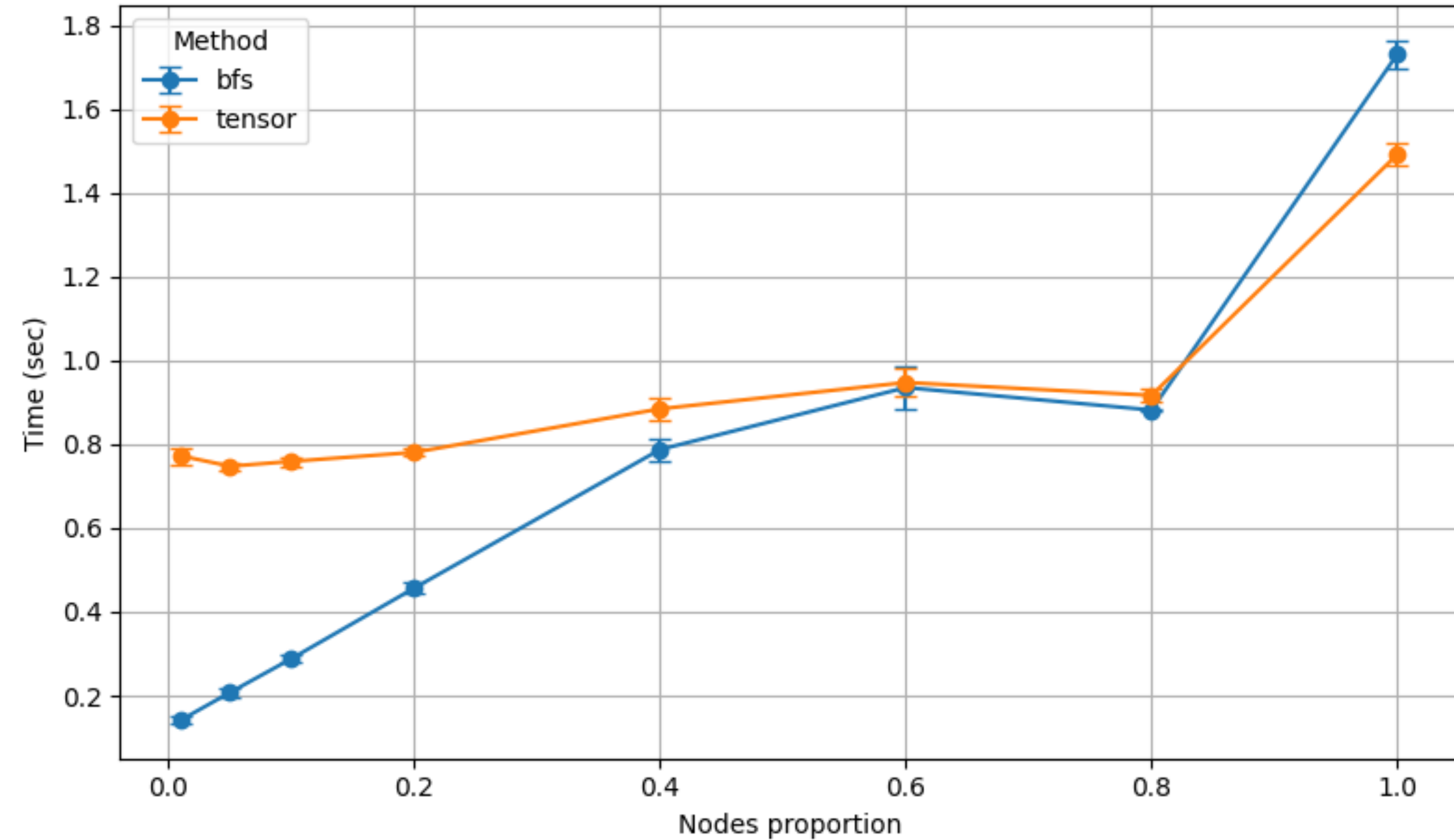
Результаты

«atom», сравнение алгоритмов

Query: type*, Matrix type: csr

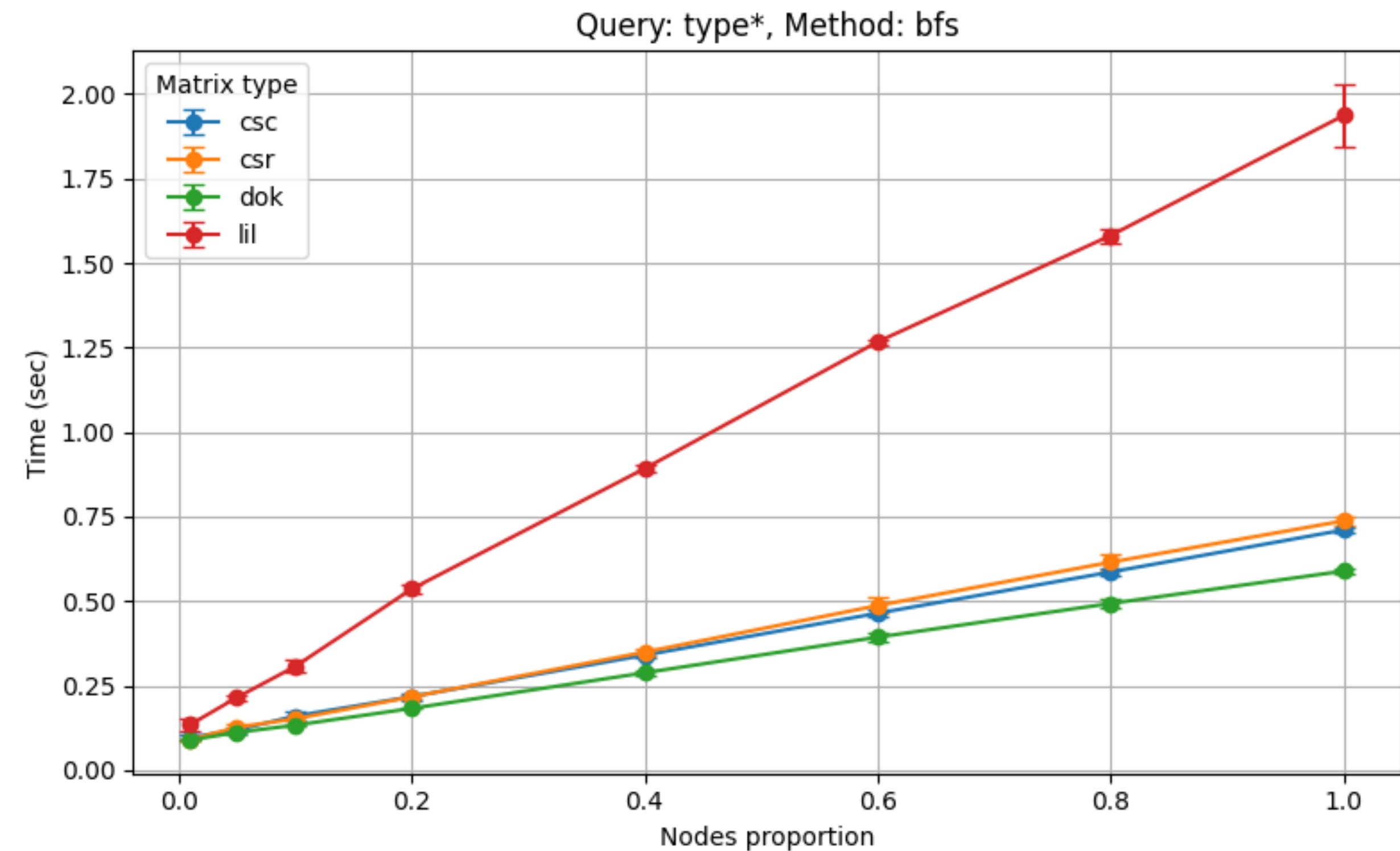
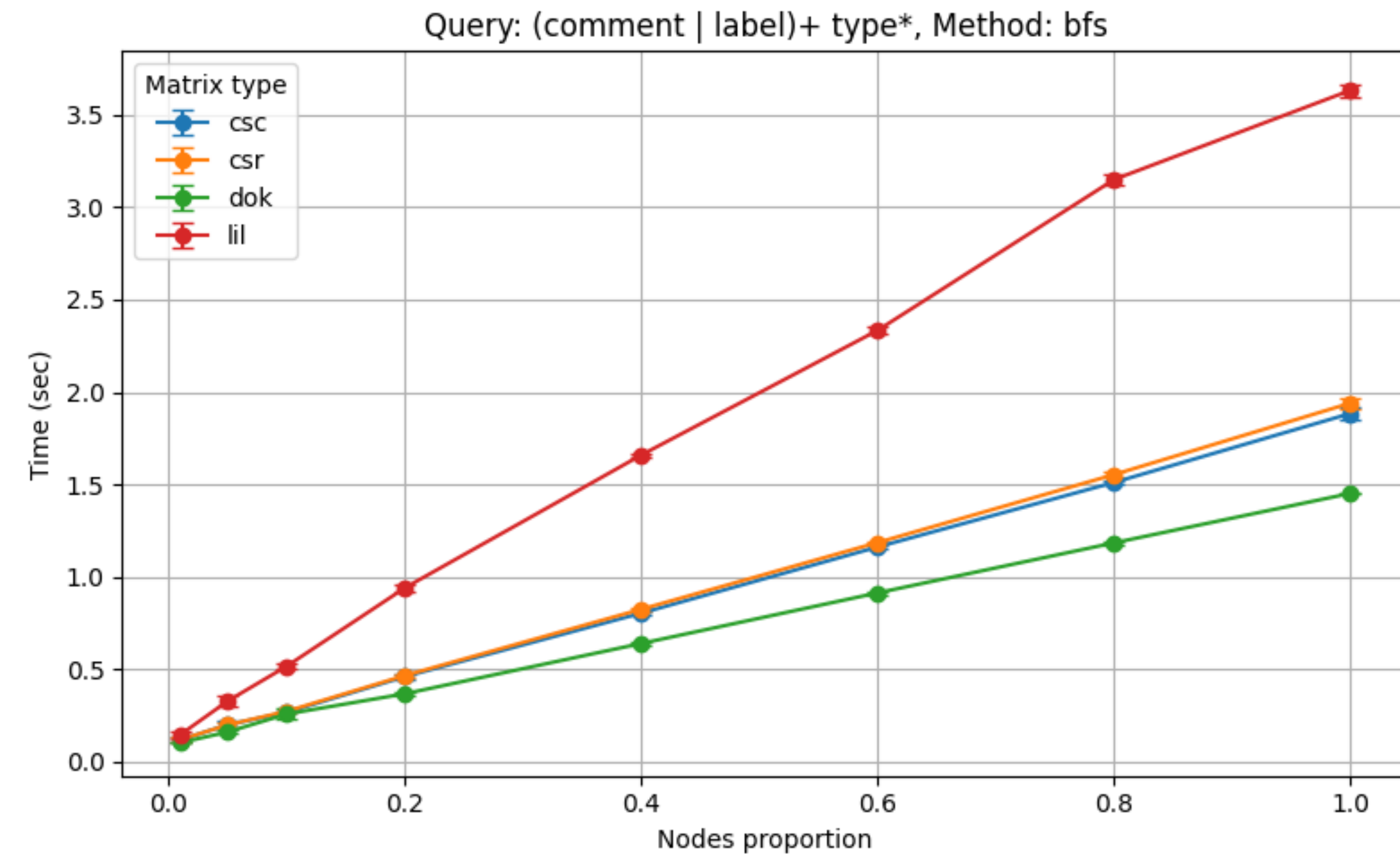


Query: (type | label) (type | subClassOf)*, Matrix type: csc



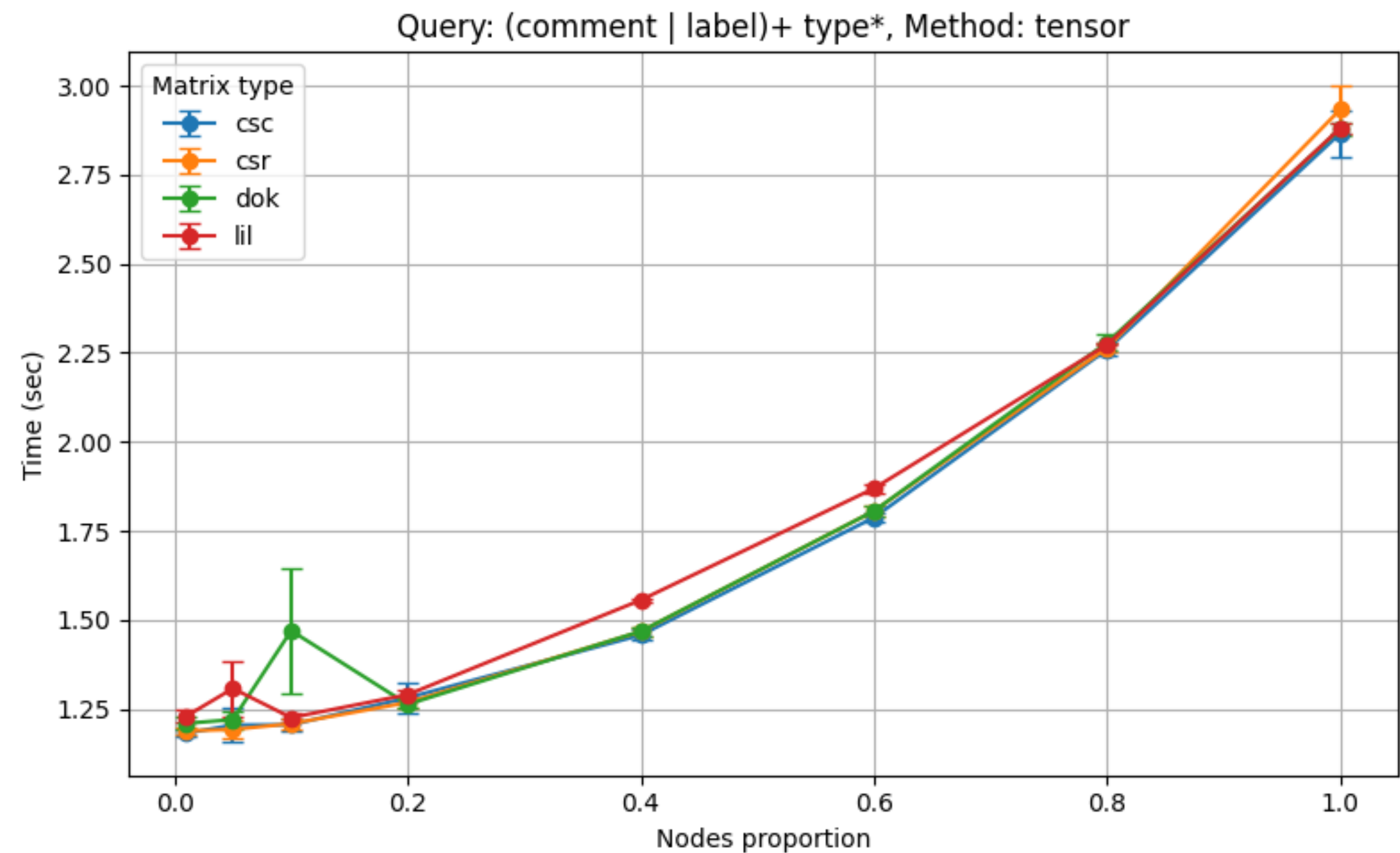
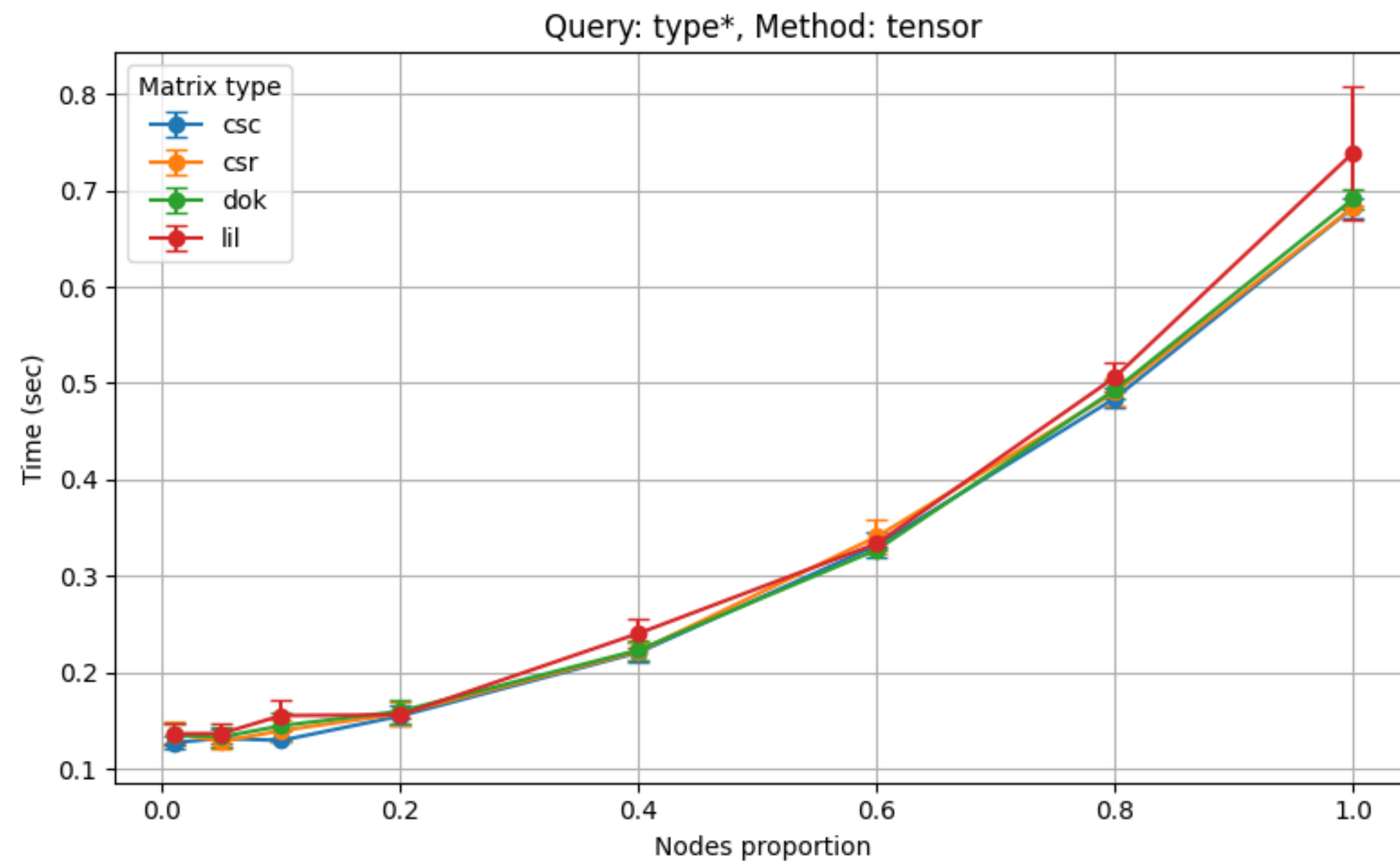
Результаты

«foaf», сравнение форматов матриц



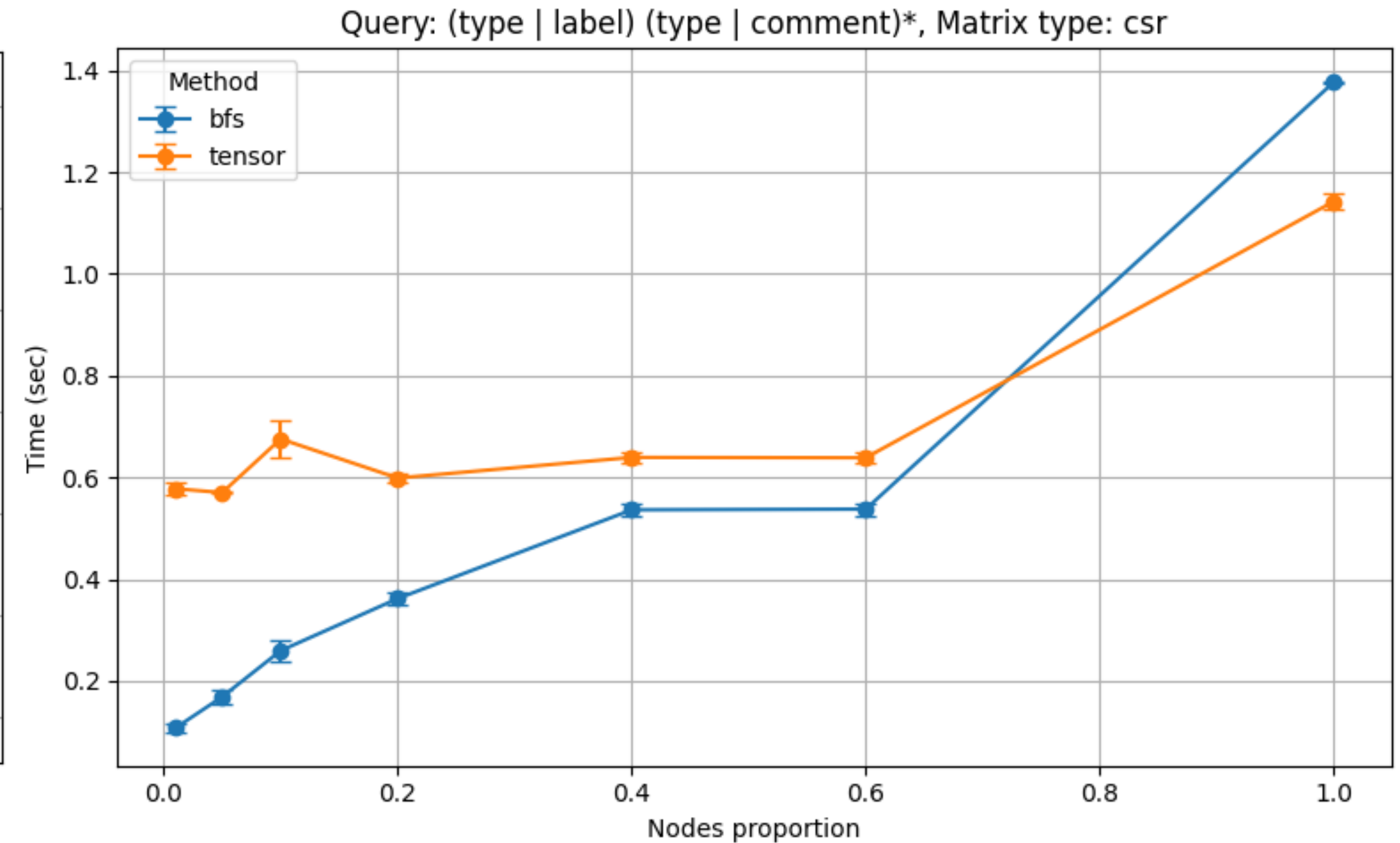
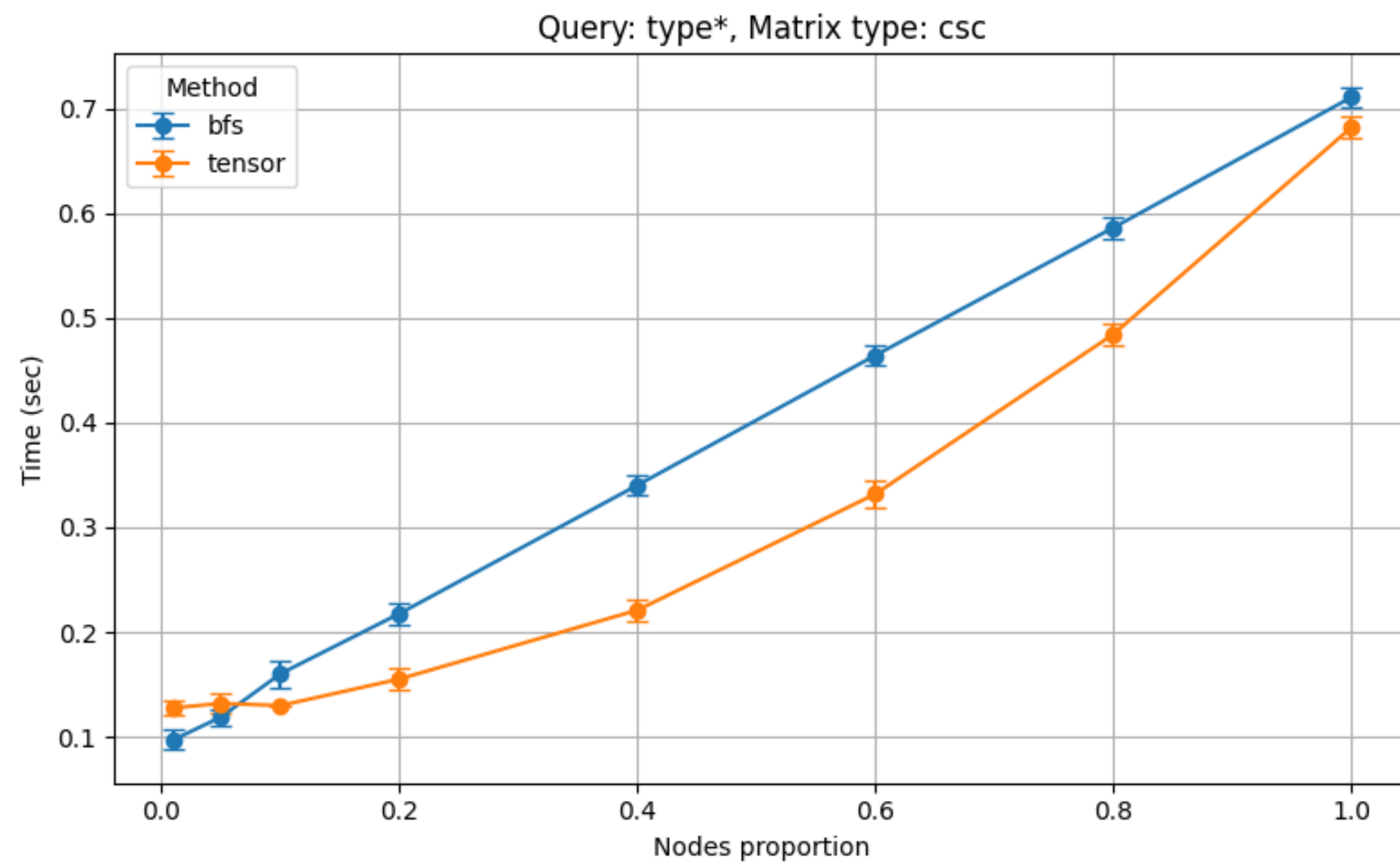
Результаты

«foaf», сравнение форматов матриц



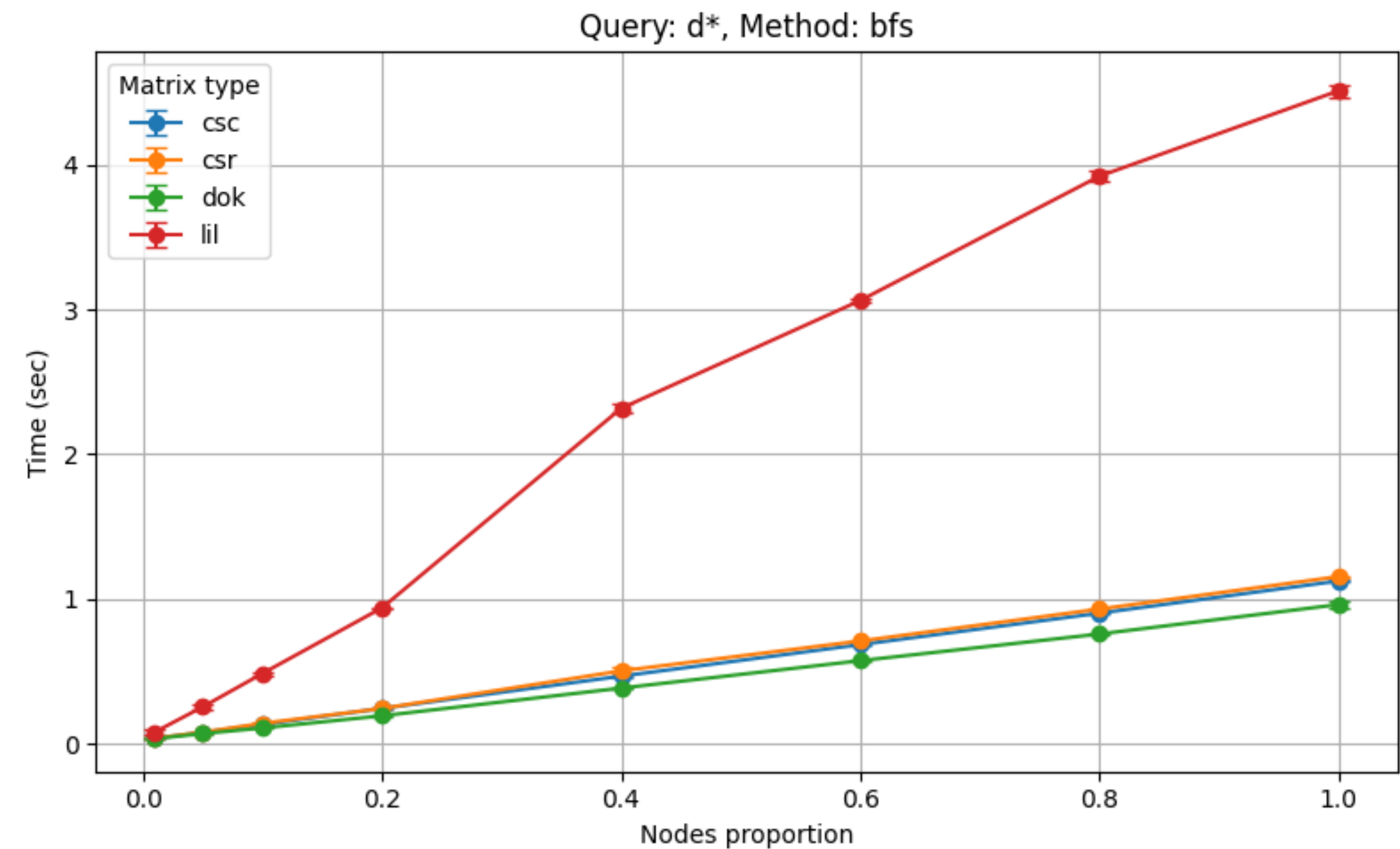
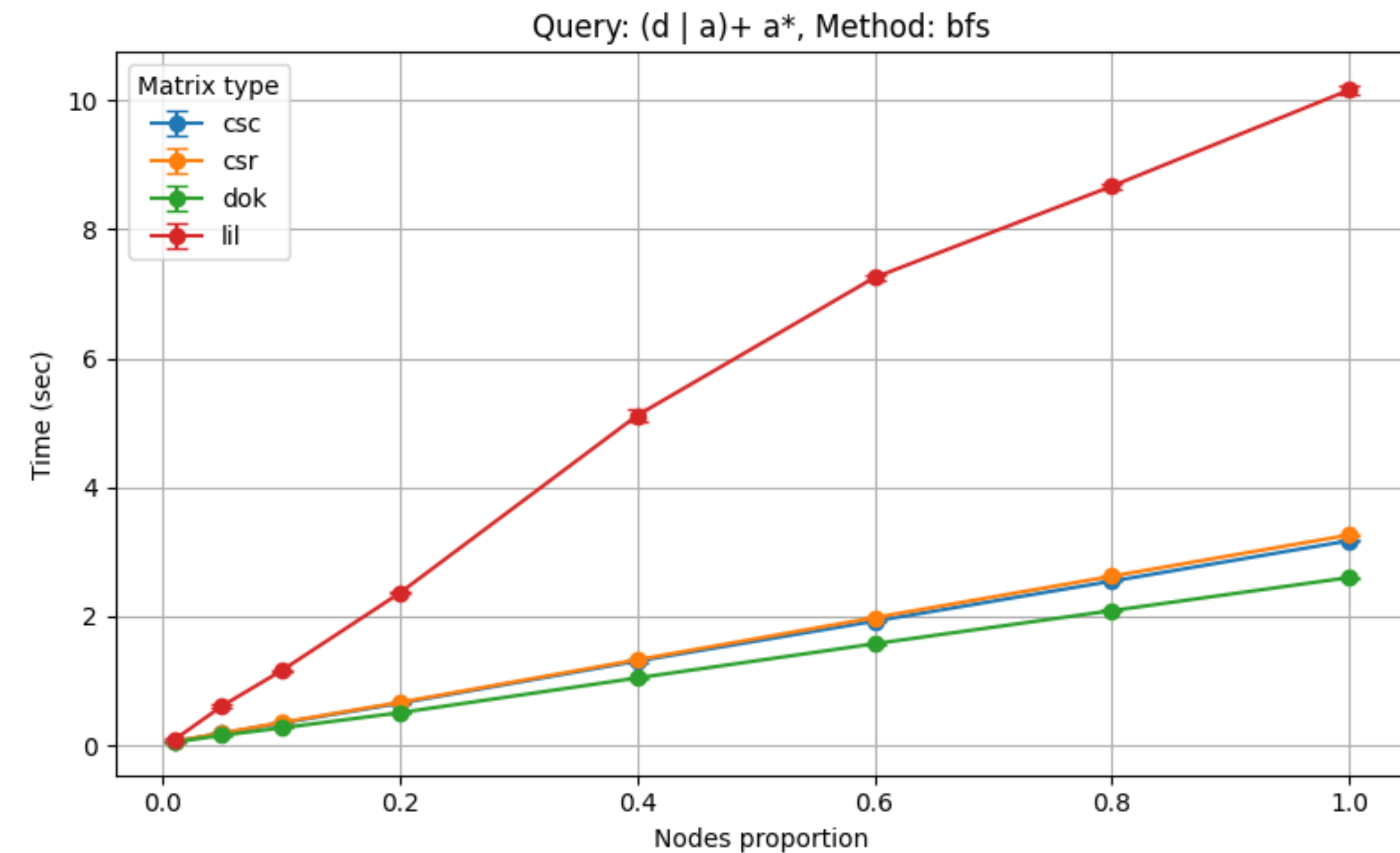
Результаты

«foaf», сравнение алгоритмов



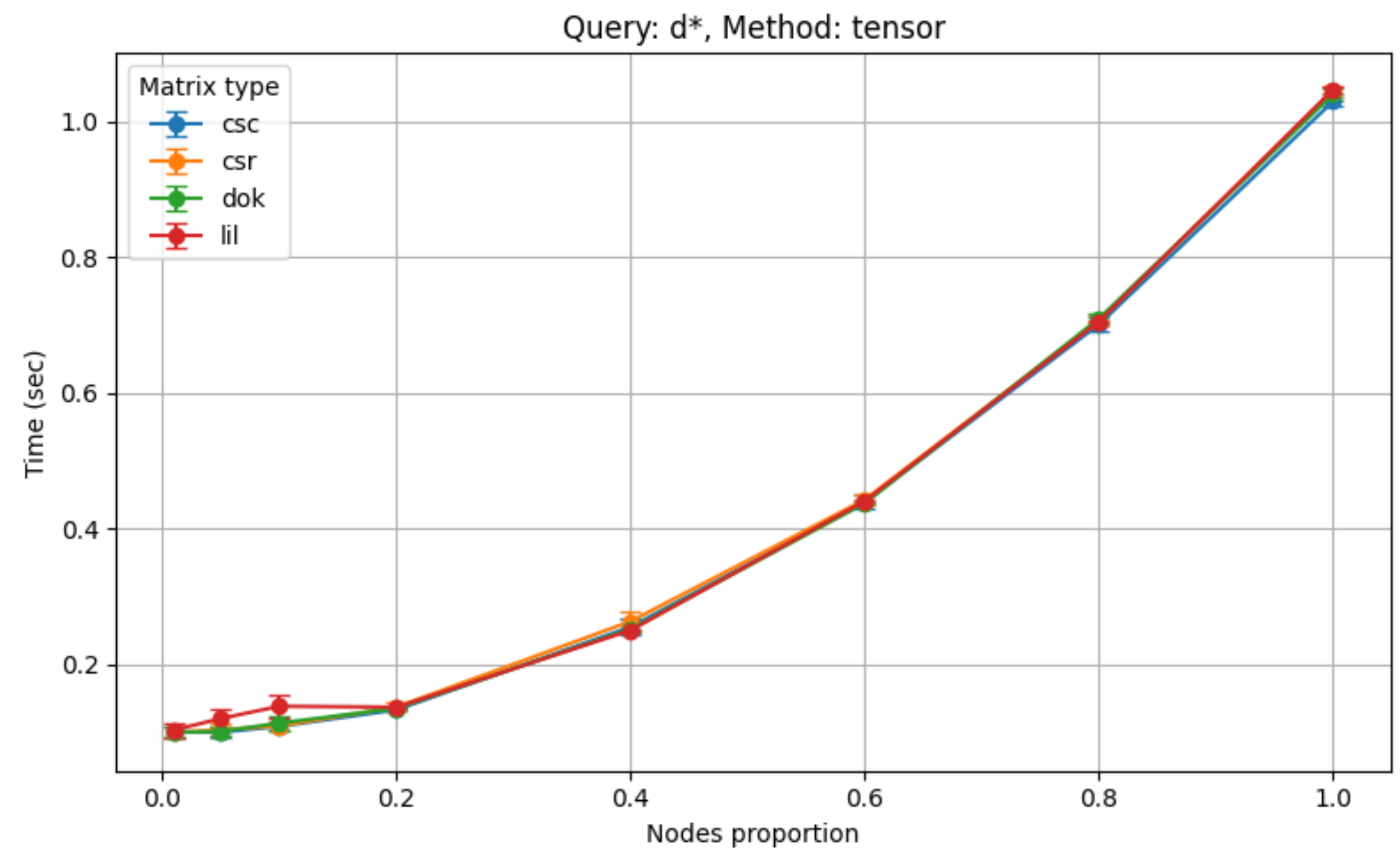
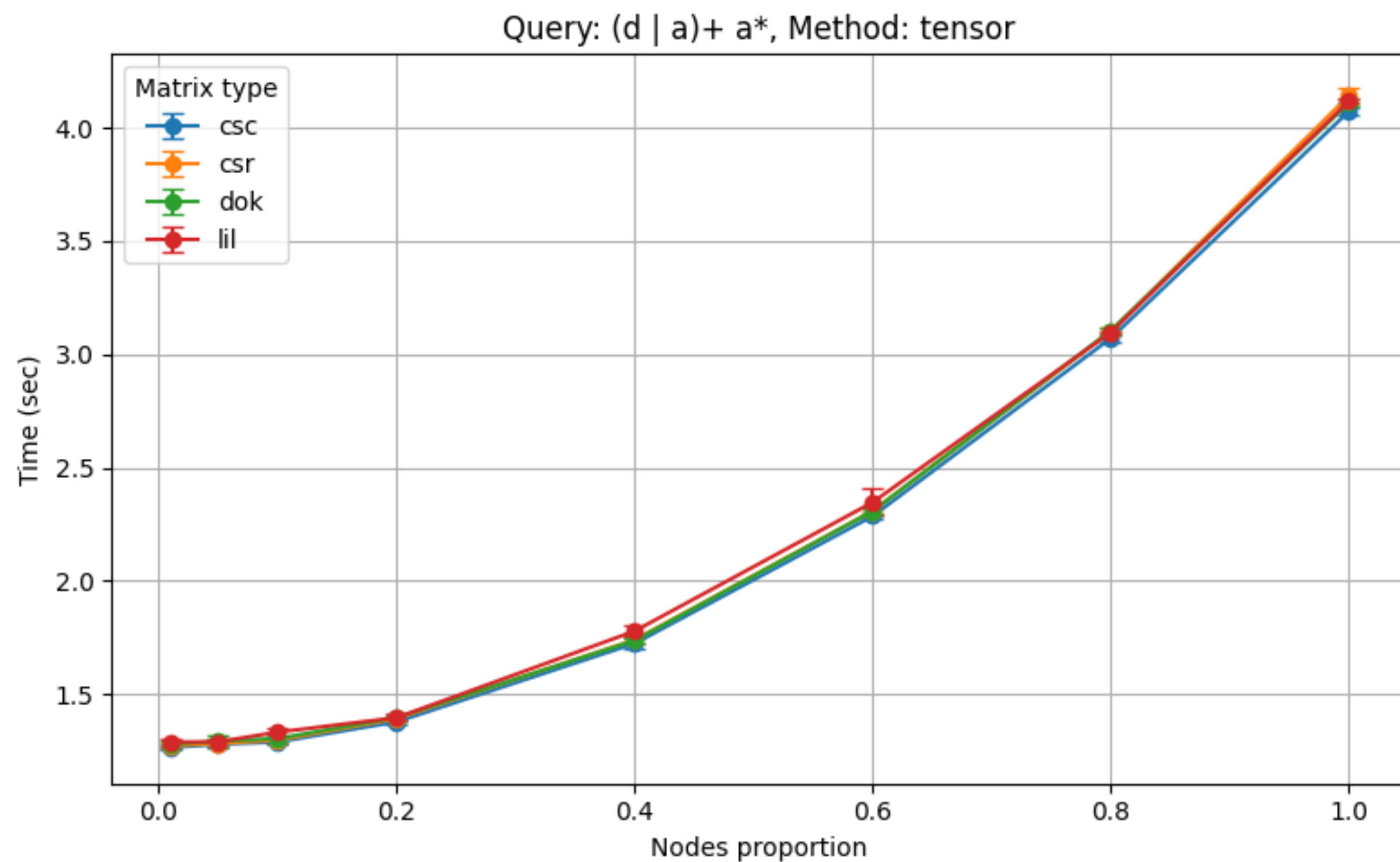
Результаты

«WC», сравнение форматов матриц



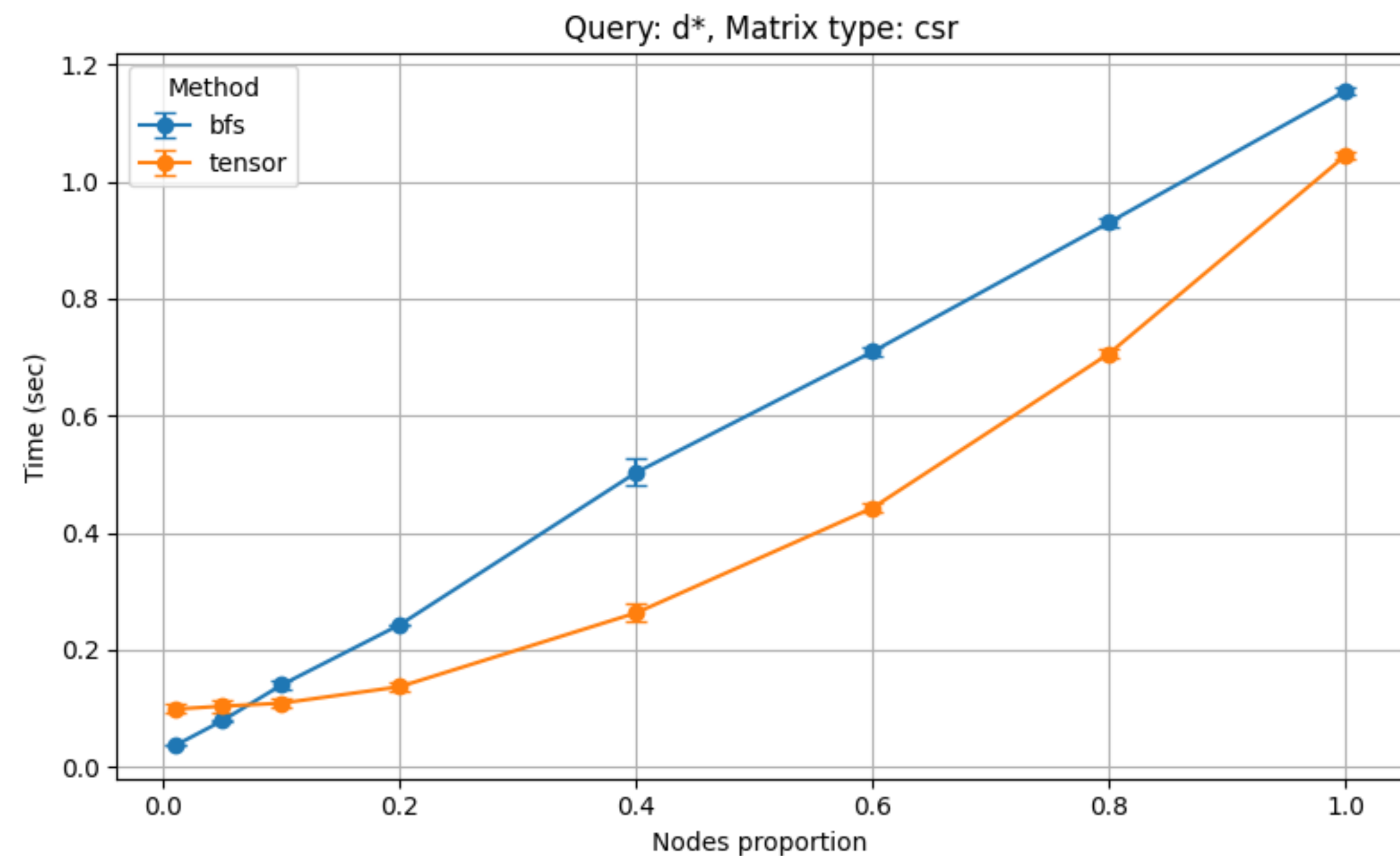
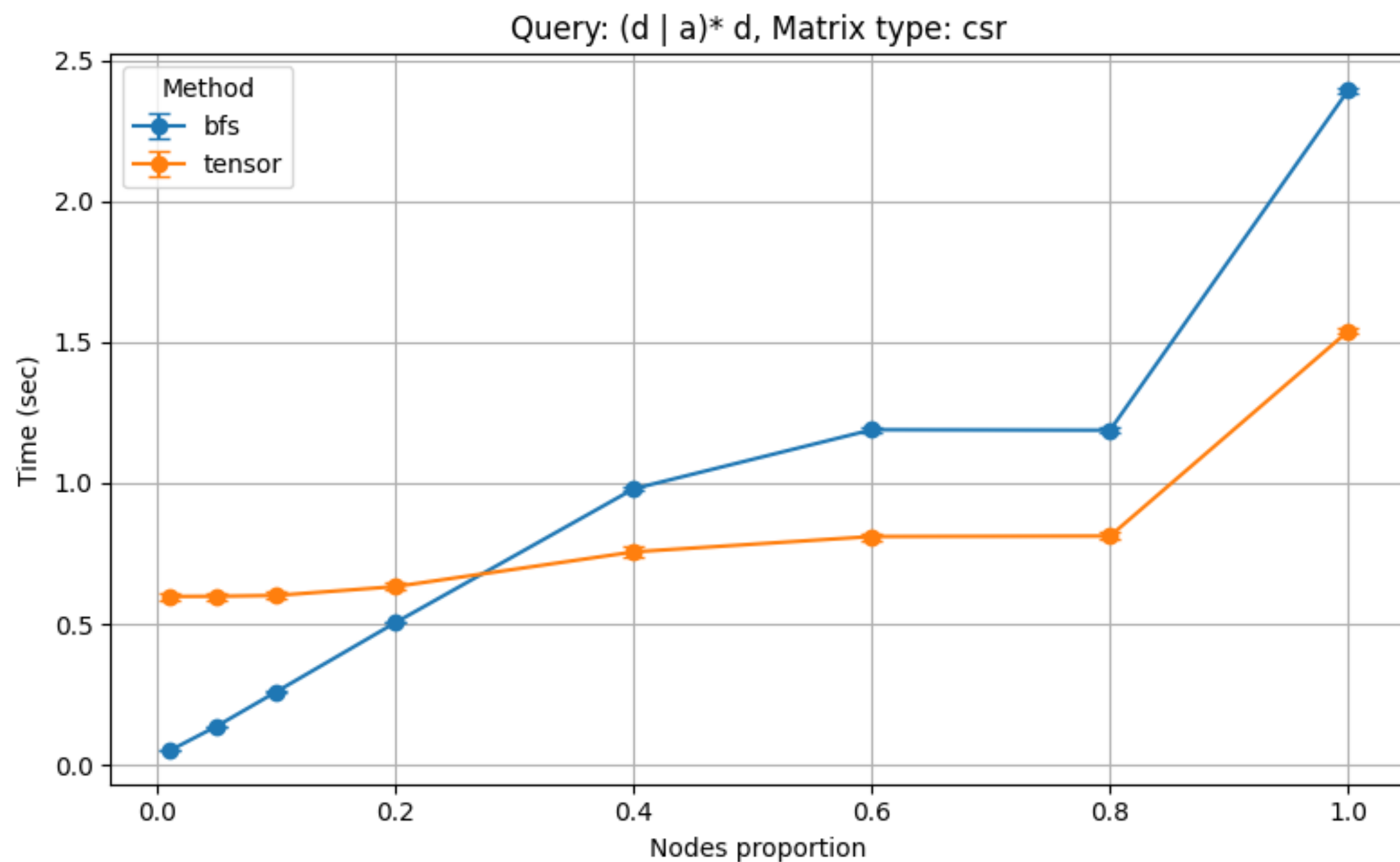
Результаты

«WC», сравнение форматов матриц



Результаты

«WC», сравнение алгоритмов

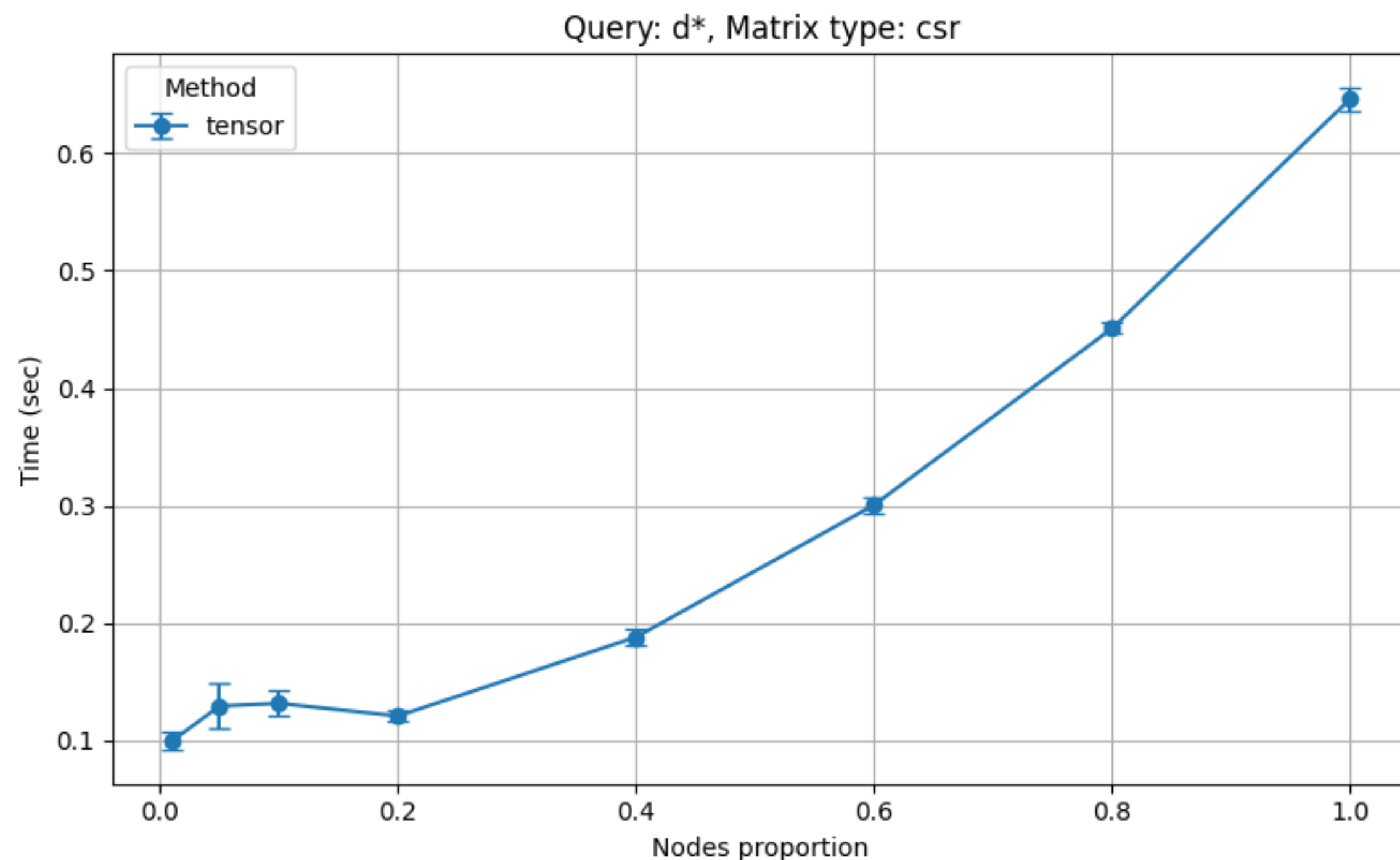


Анализ результатов

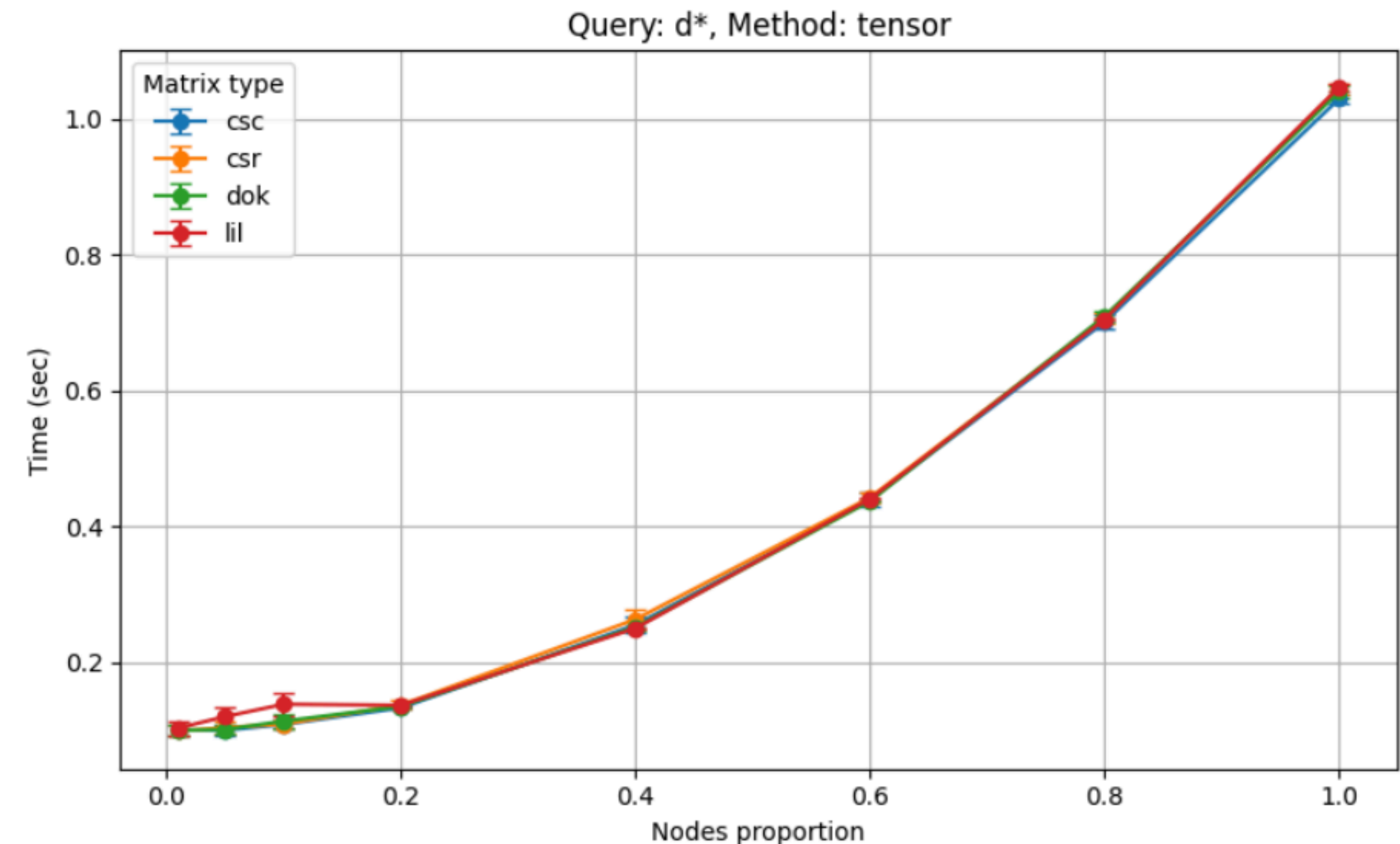
Какое представление разреженных матриц и векторов лучше подходит для каждой из решаемых задач?

- Для алгоритма на основе BFS наиболее быстрым оказался формат dok за счет более быстрого доступа к значениям
- Для тензорного алгоритма разница оказалась практически незаметной, так как `scipy.sparse` умеет преобразовывать lil и dok в csr

Разница между временем работы алгоритма с неявным приведением к CSR и с явным приведением к DOK



В этом эксперименте проводился каст матрицы пересечения к типу DOK



В этом эксперименте матрица пересечения неявно проводилась к CSR

Анализ результатов

Начиная с какого размера стартового множества выгоднее решать задачу для всех пар и выбирать нужные?

- Чем сложнее запрос, тем дольше более выгодным остается использовать BFS. Для запросов вида $l_1 * \text{тензорный алгоритм}$ начинает опережать BFS уже на 10% вершин, а на запросе $(l_1 \mid l_2) * l_1$ только на 40%